

C++ Memory Management and Object Semantics

1. Overview

C++ gives programmers direct control over memory and object lifetime. This topic covers how objects are stored, accessed, created, destroyed, copied, moved, and managed safely.

Definition

In C++, memory management means deciding:

- Where an object lives: stack or heap
- How it is accessed: pointer or reference
- How it is created: automatic object, `new`, or `malloc`
- How it is destroyed: automatic destruction, `delete`, or `free`
- How ownership is managed: RAI and smart pointers

Why It Matters

Memory mistakes are one of the most common causes of C++ bugs:

- Memory leaks
- Dangling pointers
- Double deletion
- Undefined behavior
- Crashes in production
- Poor performance due to unnecessary copies

Where It Is Used in Real Systems

These concepts are used in:

- Operating systems and device drivers
- Game engines
- Browsers
- Databases
- Trading systems
- Backend servers
- Embedded systems
- High-performance libraries

Why Interviewers Ask About It

Interviewers ask this topic because it tests:

- C++ fundamentals
- Understanding of object lifetime
- Debugging ability

- Performance awareness
- Ability to write safe production code
- Knowledge of modern C++ practices

2. Core Idea

The core idea is:

In C++, every object has a lifetime, a location in memory, and usually an owner.

If you understand ownership and lifetime, most C++ memory questions become easier.

Intuition

Think of memory like rooms in a building:

- Stack memory is like a temporary desk space. It is automatically cleaned when you leave the room.
- Heap memory is like rented storage. You must return it manually or use a manager.
- Pointers are like address slips.
- References are like permanent nicknames for existing objects.
- RAII is like hiring a responsible person who returns resources automatically.

Small Example

```
#include <iostream>
#include <memory>
#include <vector>

int main() {
    int x = 10;           // stack object
    int* p = new int(20); // heap object

    std::unique_ptr<int> q = std::make_unique<int>(30); // heap object with
    automatic cleanup

    std::cout << x << " " << *p << " " << *q << '\n';

    delete p;             // manual cleanup
}
```

Step-by-Step Explanation

1. `x` is created on the stack.
2. `p` stores the address of an integer created on the heap.
3. `q` owns a heap integer through RAII.
4. `delete p` manually releases heap memory.
5. `q` automatically releases memory when it goes out of scope.

The modern C++ lesson is simple:

Prefer automatic objects and smart pointers over raw owning pointers.

3. Important Subtopics

3.1 Pointer vs Reference

What It Means

A pointer stores the memory address of an object. A reference is an alias for an existing object.

```
int x = 10;
int* p = &x;    // pointer
int& r = x;     // reference
```

Why It Matters

Pointers and references are used for efficient parameter passing, dynamic data structures, polymorphism, and avoiding unnecessary copies.

Example

```
void incrementByPointer(int* p) {
    if (p != nullptr) {
        (*p)++;
    }
}

void incrementByReference(int& r) {
    r++;
}
```

Common Interview Angle

Interviewers often ask:

- Can a reference be null?
- Can a reference be reseated?
- When should you use pointer vs reference?

Use a reference when the object must exist. Use a pointer when null or reseating is meaningful.

3.2 Stack vs Heap

What It Means

Stack memory is automatically managed and tied to function scope. Heap memory is manually or dynamically managed and can outlive the function that created it.

```
void demo() {  
    int a = 10;           // stack  
    int* b = new int(20); // heap  
    delete b;  
}
```

Why It Matters

The stack is fast and simple, but limited in size. The heap is flexible, but slower and requires careful lifetime management.

Common Interview Angle

Interviewers expect you to know:

- Stack objects are destroyed automatically.
 - Heap objects must be released or owned by RAII wrappers.
 - Returning the address of a local stack variable is dangerous.
-

3.3 malloc vs new

What It Means

`malloc` allocates raw memory. `new` allocates memory and constructs an object.

```
int* a = (int*)malloc(sizeof(int)); // raw memory  
*a = 10;  
free(a);  
  
int* b = new int(10); // allocation + construction  
delete b;
```

Why It Matters

In C++, constructors and destructors matter. `malloc` does not call constructors. `free` does not call destructors.

Common Interview Angle

The expected answer:

- `new` calls constructor.
- `delete` calls destructor.

- `malloc` returns `void*`.
 - `malloc` must be paired with `free`.
 - `new` must be paired with `delete`.
-

3.4 `delete` vs `free`

What It Means

`delete` destroys objects created with `new`. `free` releases memory allocated with `malloc`, `calloc`, or `realloc`.

```
int* p = new int(5);
delete p;

int* q = (int*)malloc(sizeof(int));
free(q);
```

Why It Matters

Mixing them causes undefined behavior.

```
int* p = new int(10);
free(p); // wrong
```

Common Interview Angle

Interviewers ask what happens if you use `free` on memory allocated by `new`. The correct answer is undefined behavior.

3.5 RAI

What It Means

RAI stands for Resource Acquisition Is Initialization.

It means a resource is acquired in a constructor and released in a destructor.

```
#include <fstream>

void writeFile() {
    std::ofstream file("log.txt");
    file << "Hello";
} // file closes automatically here
```

Why It Matters

RAII prevents leaks even when exceptions occur.

Common Interview Angle

Interviewers expect examples like:

- `std::unique_ptr`
 - `std::shared_ptr`
 - `std::lock_guard`
 - `std::fstream`
 - `std::vector`
-

3.6 Smart Pointers

What It Means

Smart pointers are objects that manage heap memory automatically.

Main smart pointers:

- `std::unique_ptr`
- `std::shared_ptr`
- `std::weak_ptr`

Why It Matters

They reduce memory leaks and make ownership clearer.

Example

```
#include <memory>

std::unique_ptr<int> p = std::make_unique<int>(10);
```

No manual `delete` is needed.

Common Interview Angle

Interviewers ask:

- Which smart pointer should be used by default?
- What is reference counting?
- What problem does `weak_ptr` solve?

Use `unique_ptr` by default. Use `shared_ptr` only when ownership is genuinely shared.

3.7 `unique_ptr` vs `shared_ptr`

What It Means

`unique_ptr` represents exclusive ownership. `shared_ptr` represents shared ownership using reference counting.

```
auto a = std::make_unique<int>(10);

auto b = std::make_shared<int>(20);
auto c = b; // both share ownership
```

Why It Matters

Unnecessary `shared_ptr` can make ownership unclear and add runtime overhead.

Common Interview Angle

Interviewers may ask why `unique_ptr` cannot be copied. The answer: copying would create two owners for the same object, violating exclusive ownership. It can be moved.

3.8 Memory Leak

What It Means

A memory leak happens when allocated memory is no longer reachable and cannot be freed.

```
void leak() {
    int* p = new int(10);
} // p is lost, memory leaked
```

Why It Matters

Leaks can slowly increase memory usage and crash long-running systems.

Common Interview Angle

Interviewers expect you to mention:

- Forgetting `delete`
 - Exception paths
 - Cyclic `shared_ptr`
 - Tools like Valgrind, AddressSanitizer, and LeakSanitizer
-

3.9 Dangling Pointer

What It Means

A dangling pointer points to memory that is no longer valid.

```
int* p;
{
    int x = 10;
    p = &x;
}
// p is dangling here
```

Why It Matters

Using a dangling pointer causes undefined behavior.

Common Interview Angle

Interviewers often ask about:

- Returning address of local variable
 - Use-after-free
 - Iterator invalidation
 - References to destroyed objects
-

3.10 Vector Resizing

What It Means

`std::vector` stores elements contiguously. When capacity is not enough, it allocates a larger block, moves or copies elements, and releases the old block.

```
std::vector<int> v;
v.push_back(1);
v.push_back(2);
```

Why It Matters

Reallocation invalidates pointers, references, and iterators to vector elements.

Common Interview Angle

Expected points:

- `size()` is number of elements.
- `capacity()` is allocated storage.
- `reserve()` can reduce reallocations.

- `resize()` changes size.
 - Reallocation may invalidate iterators.
-

3.11 `map` vs `unordered_map`

What It Means

`std::map` is usually implemented as a balanced binary search tree. `std::unordered_map` is implemented as a hash table.

Why It Matters

They have different ordering, performance, and memory behavior.

```
std::map<int, std::string> ordered;
std::unordered_map<int, std::string> hashed;
```

Common Interview Angle

Interviewers ask:

- Which is faster on average?
 - Which keeps keys sorted?
 - What is worst-case lookup in `unordered_map`?
-

3.12 Virtual Function

What It Means

A virtual function allows runtime polymorphism. The function called depends on the actual object type, not just the pointer or reference type.

```
class Animal {
public:
    virtual void speak() {
        std::cout << "Animal\n";
    }
};

class Dog : public Animal {
public:
    void speak() override {
        std::cout << "Dog\n";
    }
};
```

Why It Matters

Virtual functions enable interface-based design and dynamic dispatch.

Common Interview Angle

Interviewers expect terms like:

- Runtime polymorphism
 - Vtable
 - Vptr
 - Dynamic dispatch
 - `override`
-

3.13 Virtual Destructor

What It Means

A base class destructor should be virtual if objects may be deleted through a base class pointer.

```
class Base {  
public:  
    virtual ~Base() = default;  
};  
  
class Derived : public Base {  
public:  
    ~Derived() {  
        // cleanup  
    }  
};
```

Why It Matters

Without a virtual destructor, deleting a derived object through a base pointer can cause undefined behavior.

```
Base* p = new Derived();  
delete p; // safe only if Base destructor is virtual
```

Common Interview Angle

If a class has virtual functions and is intended for polymorphic deletion, give it a virtual destructor.

3.14 Copy vs Move

What It Means

Copying duplicates a resource. Moving transfers ownership of a resource.

```
std::string a = "hello";  
std::string b = a;           // copy  
std::string c = std::move(a); // move
```

Why It Matters

Move semantics improve performance by avoiding expensive deep copies.

Common Interview Angle

Interviewers ask:

- What is an rvalue reference?
- What does `std::move` actually do?
- Why is move constructor useful?

Important: `std::move` does not move by itself. It casts an object to an rvalue so move operations can be used.

4. Real-World Example

Backend Server Request Handling

Imagine a C++ backend server handling thousands of requests per second.

```
class RequestContext {  
public:  
    std::vector<char> buffer;  
    std::unique_ptr<UserSession> session;  
  
    RequestContext(size_t bufferSize)  
        : buffer(bufferSize),  
          session(std::make_unique<UserSession>()) {}  
};
```

What happens:

1. `RequestContext` is created for a client request.
2. `buffer` uses heap memory internally but manages it through RAII.
3. `session` is owned by `unique_ptr`.
4. When the request finishes, destructors clean everything automatically.
5. If an exception occurs, cleanup still happens.

This is why modern C++ favors RAII and standard containers.

Browser Example

A browser may use:

- `vector` for storing tabs or DOM children
- `unordered_map` for cache lookup
- `shared_ptr` for shared resources
- `weak_ptr` to avoid ownership cycles
- Virtual functions for rendering different UI components
- Move semantics for transferring large buffers efficiently

5. Diagrams / Mental Models

Stack vs Heap

```
Function call stack
-----
main()
  x = 10
  p = address 0x500

Heap
-----
0x500 -> int value 20
```

Pointer vs Reference

```
int x = 10;

Pointer:
p -----> x

Reference:
r is another name for x
```

Vector Growth

```
Initial:
[1][2][3] capacity = 3

push_back(4)

New allocation:
[1][2][3][4][_][_] capacity = 6
```

```
Old memory is released.  
Old iterators/pointers may be invalid.
```

Smart Pointer Ownership

```
unique_ptr:  
owner -----> object  
  
shared_ptr:  
owner1 ----\  
          ---> object  reference count = 2  
owner2 ----/  
  
weak_ptr:  
observer -- observes object but does not increase reference count
```

Virtual Function Dispatch

```
Base* p = new Derived();  
p->run();  
  
Compile-time type: Base*  
Runtime object: Derived  
Called function: Derived::run()
```

6. Common Interview Questions

1. What is the difference between a pointer and a reference?

Answer

A pointer stores an address and can be null or reassigned. A reference is an alias for an existing object and must be initialized when declared.

Key Points Interviewer Expects

- Pointer can be null.
- Pointer can be reseated.
- Reference usually cannot be null.
- Reference cannot be reseated.
- Both can avoid copying.

Common Mistakes

- Saying references are always implemented differently from pointers.

- Saying references can be reassigned.
-

2. What is the difference between stack and heap memory?

Answer

Stack memory is automatically managed and tied to scope. Heap memory is dynamically allocated and must be released manually or managed by RAI.

Key Points Interviewer Expects

- Stack is fast and scope-bound.
- Heap is flexible but needs lifetime management.
- Stack has limited size.
- Heap allocation is relatively expensive.

Common Mistakes

- Saying all local variables are always stored physically on stack. Optimizers may choose differently.
 - Forgetting that standard containers may use heap internally.
-

3. What is the difference between `malloc` and `new`?

Answer

`malloc` allocates raw memory and returns `void*`. `new` allocates memory and calls the constructor.

Key Points Interviewer Expects

- `new` calls constructor.
- `malloc` does not call constructor.
- `delete` calls destructor.
- `free` does not call destructor.
- Do not mix allocation and deallocation APIs.

Common Mistakes

- Saying `malloc` initializes C++ objects.
 - Using `free` with `new`.
-

4. What happens if you call `delete` on memory allocated by `malloc`?

Answer

It causes undefined behavior. Memory allocated with `malloc` must be released with `free`.

Key Points Interviewer Expects

- Pair `new` with `delete`.
- Pair `new[]` with `delete[]`.
- Pair `malloc` with `free`.

Common Mistakes

- Saying it always crashes.
 - Saying it is okay for primitive types.
-

5. What is RAII?

Answer

RAII is a C++ technique where resources are acquired in constructors and released in destructors.

Key Points Interviewer Expects

- Resource lifetime is tied to object lifetime.
- Destructors clean up automatically.
- Exception-safe cleanup.
- Used by `vector`, `fstream`, `lock_guard`, and smart pointers.

Common Mistakes

- Thinking RAII is only about memory.
 - Forgetting file handles, locks, sockets, and database connections.
-

6. What is the difference between `unique_ptr` and `shared_ptr`?

Answer

`unique_ptr` has exclusive ownership and cannot be copied. `shared_ptr` allows multiple owners using reference counting.

Key Points Interviewer Expects

- Prefer `unique_ptr` by default.
- `shared_ptr` has overhead.
- `shared_ptr` destroys the object when reference count becomes zero.
- Cycles with `shared_ptr` can leak memory.

Common Mistakes

- Using `shared_ptr` everywhere.
 - Saying `unique_ptr` cannot be passed to functions at all. It can be moved or passed by reference.
-

7. What is a memory leak?

Answer

A memory leak happens when allocated memory is no longer reachable and cannot be released.

Key Points Interviewer Expects

- Common with forgotten `delete`.
- Common in exception paths.
- Long-running programs suffer more.
- RAII helps prevent leaks.

Common Mistakes

- Thinking memory leak means immediate crash.
 - Ignoring cyclic `shared_ptr`.
-

8. What is a dangling pointer?

Answer

A dangling pointer points to memory whose lifetime has ended.

Key Points Interviewer Expects

- Use-after-free.
- Address of local variable after function returns.
- Invalidated vector iterators.
- Undefined behavior if dereferenced.

Common Mistakes

- Saying setting one pointer to `nullptr` fixes all aliases.
 - Confusing dangling pointer with memory leak.
-

9. How does `std::vector` resizing work?

Answer

When a vector needs more capacity, it allocates a larger memory block, moves or copies existing elements, destroys old elements, and releases old storage.

Key Points Interviewer Expects

- `size()` vs `capacity()`.
- `reserve()` controls capacity.
- `resize()` changes number of elements.
- Reallocation invalidates iterators and references.

Common Mistakes

- Confusing `reserve()` and `resize()`.
 - Assuming vector never moves elements.
-

10. What is the difference between `map` and `unordered_map`?

Answer

`map` stores keys in sorted order and usually uses a balanced tree. `unordered_map` uses hashing and does not maintain sorted order.

Key Points Interviewer Expects

- `map`: $O(\log n)$ operations.
- `unordered_map`: $O(1)$ average, $O(n)$ worst case.
- `map` supports ordered traversal.
- `unordered_map` needs hash function and equality comparison.

Common Mistakes

- Saying `unordered_map` is always faster.
 - Forgetting worst-case hash collisions.
-

11. What is a virtual function?

Answer

A virtual function enables runtime polymorphism. When called through a base pointer or reference, the derived class implementation can be invoked.

Key Points Interviewer Expects

- Dynamic dispatch.
- Base class pointer/reference.
- Vtable and vptr concept.
- Use `override` in derived classes.

Common Mistakes

- Confusing function overloading with overriding.
 - Forgetting virtual dispatch needs pointer or reference behavior.
-

12. Why should a base class destructor be virtual?

Answer

If an object may be deleted through a base class pointer, the base destructor should be virtual so the derived destructor runs correctly.

Key Points Interviewer Expects

- Safe polymorphic deletion.
- Derived destructor must run.
- Prevents undefined behavior.

Common Mistakes

- Saying every class must have a virtual destructor.
 - Forgetting that virtual destructors add polymorphic overhead.
-

13. What is the difference between copy and move?

Answer

Copy duplicates an object or resource. Move transfers ownership of resources from one object to another.

Key Points Interviewer Expects

- Move avoids expensive deep copies.
- Move uses rvalue references.
- `std::move` is a cast.
- Moved-from object remains valid but unspecified.

Common Mistakes

- Thinking `std::move` physically moves data by itself.
 - Using moved-from objects as if unchanged.
-

7. Deep-Dive Questions

1. Why can cyclic `shared_ptr` cause memory leaks?

If two objects own each other using `shared_ptr`, their reference counts may never reach zero.

```
struct B;

struct A {
    std::shared_ptr<B> b;
};

struct B {
    std::shared_ptr<A> a;
};
```

Even if external references are gone, **A** and **B** keep each other alive. Use `weak_ptr` for one side of the relationship.

2. What happens when a vector stores objects with move constructors?

During reallocation, `vector` may move elements instead of copying them. If the move constructor is marked `noexcept`, vector can safely prefer moving in many cases.

```
class Buffer {
public:
    Buffer(Buffer&& other) noexcept {
        // transfer resource
    }
};
```

This improves performance for objects that own large resources.

3. Why is deleting through a non-virtual base destructor dangerous?

```
class Base {
public:
    ~Base() {}
};

class Derived : public Base {
public:
    int* data = new int[100];
    ~Derived() {
        delete[] data;
    }
};

Base* p = new Derived();
delete p; // undefined behavior
```

The derived destructor may not run, so resources owned by `Derived` may leak.

4. What is the Rule of Three, Rule of Five, and Rule of Zero?

Rule of Three: if a class defines any of destructor, copy constructor, or copy assignment, it probably needs all three.

Rule of Five: in modern C++, also consider move constructor and move assignment.

Rule of Zero: prefer designing classes so they do not manually manage resources. Use standard containers and smart pointers, so compiler-generated operations are correct.

Best interview answer: prefer Rule of Zero, use Rule of Five only when writing resource-owning low-level classes.

5. How does virtual dispatch usually work internally?

Most implementations use:

- A vtable per polymorphic class
- A vptr inside each polymorphic object
- Function pointers stored in the vtable

When a virtual function is called, the program looks up the correct function through the vptr.

This is implementation-dependent, but the vtable model is the common mental model expected in interviews.

8. Comparison Tables

Pointer vs Reference

Feature	Pointer	Reference
Stores address	Yes	Usually implemented as alias/address internally
Can be null	Yes	Not normally
Must be initialized	No	Yes
Can be reseated	Yes	No
Syntax to access	<code>*p, p->x</code>	Direct, <code>r.x</code>
Best use	Optional object, dynamic memory, arrays	Required object, parameter alias

Stack vs Heap

Feature	Stack	Heap
Lifetime	Scope-based	Programmer/owner controlled
Speed	Very fast	Slower
Size	Limited	Larger
Cleanup	Automatic	Manual or RAII
Common use	Local variables	Dynamic objects, large data, shared lifetime
Risk	Stack overflow	Leaks, fragmentation, dangling pointers

`malloc` vs `new`

Feature	<code>malloc</code>	<code>new</code>
Language	C library	C++ operator
Return type	<code>void*</code>	Typed pointer
Constructor called	No	Yes
Failure behavior	Returns <code>NULL</code>	Throws <code>std::bad_alloc</code> by default
Deallocation	<code>free</code>	<code>delete</code>
Use in modern C++	Rare	Rare directly; prefer smart pointers/containers

`delete` vs `free`

Feature	<code>delete</code>	<code>free</code>
Used with	<code>new</code>	<code>malloc</code> , <code>calloc</code> , <code>realloc</code>
Calls destructor	Yes	No
Type aware	Yes	No
Array version	<code>delete[]</code>	Same <code>free</code>
Mixing allowed	No	No

`unique_ptr` vs `shared_ptr`

Feature	<code>unique_ptr</code>	<code>shared_ptr</code>
Ownership	Exclusive	Shared
Copyable	No	Yes
Movable	Yes	Yes
Overhead	Low	Reference count overhead
Destruction	When owner dies	When last owner dies
Best use	Default heap ownership	Truly shared lifetime
Risk	Use after move	Cycles, unclear ownership

`map` vs `unordered_map`

Feature	<code>map</code>	<code>unordered_map</code>
Internal structure	Balanced tree	Hash table
Ordering	Sorted by key	No sorted order
Average lookup	$O(\log n)$	$O(1)$
Worst lookup	$O(\log n)$	$O(n)$

Feature	<code>map</code>	<code>unordered_map</code>
Memory overhead	Tree nodes	Buckets and nodes
Best use	Ordered traversal, range queries	Fast lookup by key

Copy vs Move

Feature	Copy	Move
Meaning	Duplicate resource/value	Transfer resource
Source after operation	Unchanged	Valid but unspecified
Cost	Can be expensive	Usually cheap
Syntax	<code>T b = a;</code>	<code>T b = std::move(a);</code>
Used for	Independent duplicate	Ownership transfer

Virtual vs Non-Virtual Function

Feature	Non-virtual	Virtual
Binding	Compile time	Runtime
Polymorphism	Static	Dynamic
Overhead	Usually none	Vtable lookup
Override behavior	Hidden unless virtual	Runtime override
Use case	Normal methods	Interface-like behavior

9. Common Mistakes

- Mixing `new` with `free`.
- Mixing `malloc` with `delete`.
- Forgetting `delete[]` for arrays allocated with `new[]`.
- Returning the address of a local variable.
- Assuming `std::move` always moves data immediately.
- Using `shared_ptr` when `unique_ptr` is enough.
- Forgetting cyclic `shared_ptr` leaks.
- Confusing `vector::reserve()` with `vector::resize()`.
- Keeping pointers or iterators to vector elements after reallocation.
- Deleting derived objects through base pointers without virtual destructors.
- Thinking references can be resealed.
- Thinking `unordered_map` is always O(1).
- Writing manual memory management when standard containers would solve the problem.

10. Edge Cases / Special Cases

delete nullptr Is Safe

```
int* p = nullptr;  
delete p; // safe
```

delete vs delete[]

```
int* a = new int[5];  
delete[] a; // correct
```

Using `delete a` here is undefined behavior.

Moved-From Object

```
std::string s = "hello";  
std::string t = std::move(s);
```

After this, `s` is still valid, but its value is unspecified.

Vector Iterator Invalidation

```
std::vector<int> v = {1, 2, 3};  
int* p = &v[0];  
v.push_back(4); // may reallocate  
// p may now be dangling
```

Reference Lifetime Extension

```
const std::string& s = std::string("hello");
```

The temporary string's lifetime is extended to match the reference lifetime.

Object Slicing

```
class Base {};  
class Derived : public Base {};  
  
Derived d;  
Base b = d; // derived part is sliced
```

Use references or pointers for polymorphic behavior.

`shared_ptr` and Raw Pointer Danger

```
int* raw = new int(10);
std::shared_ptr<int> a(raw);
std::shared_ptr<int> b(raw); // dangerous: two control blocks
```

This can cause double deletion. Prefer `make_shared`.

11. How to Explain in Interview

C++ gives direct control over memory and object lifetime. Stack objects are automatically destroyed when they go out of scope, while heap objects need explicit ownership management. Modern C++ uses RAII, standard containers, and smart pointers to avoid leaks and dangling pointers. I use `unique_ptr` for exclusive ownership, `shared_ptr` only for shared ownership, and avoid raw owning pointers. For polymorphism, I use virtual functions and make the base destructor virtual when deleting through base pointers is possible.

12. Quick Revision Notes

Key Definitions

- Pointer: variable that stores an address.
- Reference: alias for an existing object.
- Stack: scope-based automatic memory.
- Heap: dynamic memory with flexible lifetime.
- RAII: resource cleanup through destructors.
- Memory leak: allocated memory that cannot be freed.
- Dangling pointer: pointer to invalid memory.
- Virtual function: function resolved at runtime.
- Move: transfer resource ownership.

Important Points

- Prefer stack objects when possible.
- Prefer RAII over manual cleanup.
- Prefer `make_unique` and `make_shared`.
- Use `unique_ptr` by default.
- Use `weak_ptr` to break `shared_ptr` cycles.
- Use virtual destructors in polymorphic base classes.
- `vector` reallocation invalidates references, pointers, and iterators.

Common Comparisons

- Pointer vs reference
- Stack vs heap

- `malloc` vs `new`
- `delete` vs `free`
- `unique_ptr` vs `shared_ptr`
- `map` vs `unordered_map`
- Copy vs move
- Virtual vs non-virtual function

Must-Remember Facts

- `new` calls constructor.
- `delete` calls destructor.
- `malloc` and `free` do not call constructors or destructors.
- `std::move` is a cast, not a move operation by itself.
- Moved-from objects are valid but unspecified.
- `unordered_map` has O(1) average lookup, not guaranteed O(1).
- Deleting through a base pointer requires a virtual destructor.

Interview Traps

- `free(new int)` is undefined behavior.
- `delete malloc(...)` is undefined behavior.
- Returning `&localVariable` creates a dangling pointer.
- `reserve()` does not change vector size.
- `resize()` changes vector size.
- `shared_ptr` cycles leak memory.

13. Practice Tasks

Task 1: Pointer vs Reference

Write two functions:

```
void updateByPointer(int* p);  
void updateByReference(int& r);
```

Call both and explain when pointer is better than reference.

Task 2: Stack vs Heap

Write a function that creates:

- One stack integer
- One heap integer using `new`
- One heap integer using `unique_ptr`

Explain their lifetimes.

Task 3: RAIL Class

Implement a small class that owns a dynamic array and releases it in the destructor.

Then rewrite it using `std::vector<int>`.

Task 4: Smart Pointer Ownership

Create a `unique_ptr` and transfer ownership to another function using `std::move`.

Explain why copying is not allowed.

Task 5: `shared_ptr` Cycle

Create two classes that point to each other using `shared_ptr`.

Then fix the leak using `weak_ptr`.

Task 6: Vector Resizing

Write code that prints vector `size()` and `capacity()` after each `push_back`.

Then use `reserve()` and compare the output.

Task 7: Iterator Invalidation

Store a pointer to `v[0]`, call `push_back` many times, and explain why the pointer may become invalid.

Task 8: `map` vs `unordered_map`

Store student roll numbers and names in both containers.

Print the order of traversal and compare.

Task 9: Virtual Function

Create a base class `Shape` with a virtual function `area()`.

Implement `Circle` and `Rectangle`, then call `area()` through `Shape*`.

Task 10: Virtual Destructor

Create a base and derived class where the derived class allocates memory.

Delete through a base pointer and observe why the base destructor should be virtual.

Task 11: Copy vs Move

Create a class with logging inside:

- Constructor
- Copy constructor
- Move constructor
- Destructor

Push objects into a vector and observe which operations happen.

14. Final Cheat Sheet

Core Definition

C++ memory management is about controlling object lifetime, ownership, allocation, deallocation, copying, moving, and polymorphic behavior safely and efficiently.

Why It Matters

It prevents leaks, crashes, undefined behavior, and performance issues in real-world C++ systems.

Most Asked Questions

- Pointer vs reference
- Stack vs heap
- `malloc` vs `new`
- `delete` vs `free`
- What is RAII?
- `unique_ptr` vs `shared_ptr`
- What is a memory leak?
- What is a dangling pointer?
- How does vector resizing work?
- `map` vs `unordered_map`
- What is a virtual function?
- Why virtual destructor?
- Copy vs move

Common Comparisons

Comparison	One-Line Answer
Pointer vs Reference	Pointer stores an address; reference is an alias.
Stack vs Heap	Stack is automatic and scope-bound; heap is dynamic and ownership-controlled.
<code>malloc</code> vs <code>new</code>	<code>malloc</code> allocates raw memory; <code>new</code> constructs objects.
<code>delete</code> vs <code>free</code>	<code>delete</code> calls destructors; <code>free</code> only releases raw memory.
<code>unique_ptr</code> vs <code>shared_ptr</code>	<code>unique_ptr</code> has one owner; <code>shared_ptr</code> has multiple owners.
<code>map</code> vs <code>unordered_map</code>	<code>map</code> is ordered; <code>unordered_map</code> is hash-based.
Copy vs Move	Copy duplicates; move transfers resources.

One-Line Interview Answer

In modern C++, memory should be managed through object lifetime: use stack objects, RAII, standard containers, and smart pointers so resources are acquired safely and released automatically, while

understanding raw pointers, heap allocation, virtual destructors, and move semantics for performance and correctness.